

Multi-Scale Optimization Analysis

1. Experiment Overview

This assignment optimized a CIFAR-10 classifier for cloud, edge, and microcontroller deployment. Experiments used TensorFlow 2.15.1 in WSL, with an NVIDIA RTX 4070 Ti for GPU training. The data contained 45,000 training images, 5,000 validation images, and 10,000 test images. The Part 1 baseline had 324,394 parameters and reached 87.71% test accuracy.

Part 2 compared cloud training strategies. Part 3 tested pruning, quantization, and lightweight architectures. Part 4 combined these results into a pipeline that selected and exported a model for each deployment target. No physical edge board was available, so edge latency was measured on the local CPU.

2. Multi-Scale Trade-offs

The following table shows the final Part 4 selections. MiB and KiB use powers of 1,024.

Metric	Cloud server	Edge device	Microcontroller
Selected strategy	Batch 256 model	Dynamic range quantization	Lightweight architecture + INT8
Model size	1.314 MiB	0.324 MiB	35.40 KiB
Test accuracy	86.42%	87.67%	70.53%
Local reference latency	0.860 ms, GPU	0.310 ms, CPU	0.049 ms, CPU
Inference memory usage	Not measured	Not measured	Not measured
Power estimate	Unavailable	Unavailable	Unavailable
Assigned power budget	50 W	2,000 mW	10 mW
Development complexity	High	Medium	High

Cloud latency was reused from Part 2. Edge and microcontroller results used single-thread TFLite inference with batch size 1, 20 warm-up runs, and 100 timed runs. Timing includes input/output handling. GPU and CPU measurements are not a cross-device speed comparison. Cloud storage is a Keras archive, while the other files are TFLite models.

All three models met the pipeline's file-size and local reference latency limits. Actual target-device latency, inference memory, and power remain unverified. The power budgets are assignment settings, not measured consumption. The cloud model's measured training allocator peak was 745.04 MiB; this is not inference memory. Part 3's file-size-based RAM and assumed-power energy calculations do not establish hardware feasibility.

The edge model retained nearly all baseline accuracy while reducing TFLite storage by 73.81%. The microcontroller model reduced storage by 97.21%, but lost 17.18 percentage points of accuracy. These two

models form the accuracy–file-size Pareto frontier among the final selections. The cloud model was selected for training throughput rather than this two-metric objective.

```
"cloud_server": {
  "model_path": "/mnt/d/work/cloud-repository/Lec/MLS/assignment/as
  "accuracy": 0.8641999959945679,
  "model_size_mb": 1.3141555786132812,
  "estimated_latency_ms": 0.8600425800068479,
  "memory_usage_mb": null,
  "optimization_strategy": "Reused cloud configuration: batch256",
"edge_device": {
  "model_path": "/mnt/d/work/cloud-repository/l
  "accuracy": 0.8767,
  "model_size_mb": 0.32415008544921875,
  "estimated_latency_ms": 0.30998735019238666,
  "memory_usage_mb": null,
  "optimization_strategy": "dynamic_range",
"microcontroller": {
  "model_path": "/mnt/d/work/cloud-repository/Lec/MLS/assi
  "accuracy": 0.7053,
  "model_size_mb": 0.03456878662109375,
  "estimated_latency_ms": 0.04893103014182998,
  "memory_usage_mb": null,
  "optimization_strategy": "architecture_optimized_int8",
```

Figure 1. Final results for the three deployment targets.

3. Cloud Optimization Effectiveness

Mixed precision. Against the Part 2 Float32 reference, accuracy changed from 87.35% to 87.51%, and peak training memory fell from 410.56 to 378.98 MiB. However, training time increased from 141.74 to 241.56 seconds. Mixed precision saved 7.69% memory but did not accelerate this model in the tested environment.

Distributed training. Two logical GPUs shared one physical GPU. Their throughput was 4,452.4 images/s, compared with 16,702.8 for one replica. The throughput ratio was 0.267, giving 13.33% simulated two-replica efficiency. This verified synchronization but did not demonstrate real multi-GPU scaling.

Batch processing. Batch size 256 increased throughput from 17,690.1 to 25,285.4 images/s, an improvement of 42.94%, while accuracy fell from 87.35% to 86.42%. Peak memory rose to 745.04 MiB. Gradient accumulation with four micro-batches of 64 reduced memory to 319.23 MiB, but training took 307.73 seconds instead of 104.63 seconds. BatchNorm still used micro-batch statistics. Prefetching and parallel mapping showed no clear speed advantage over the serial pipeline in this run.

Knowledge distillation. The teacher achieved 88.55% accuracy. The distilled student achieved 87.86%, compared with 87.35% for ordinary student training, while retaining 324,394 parameters. This improved accuracy by 0.51 percentage points without increasing deployed size, but required teacher training and additional student training.

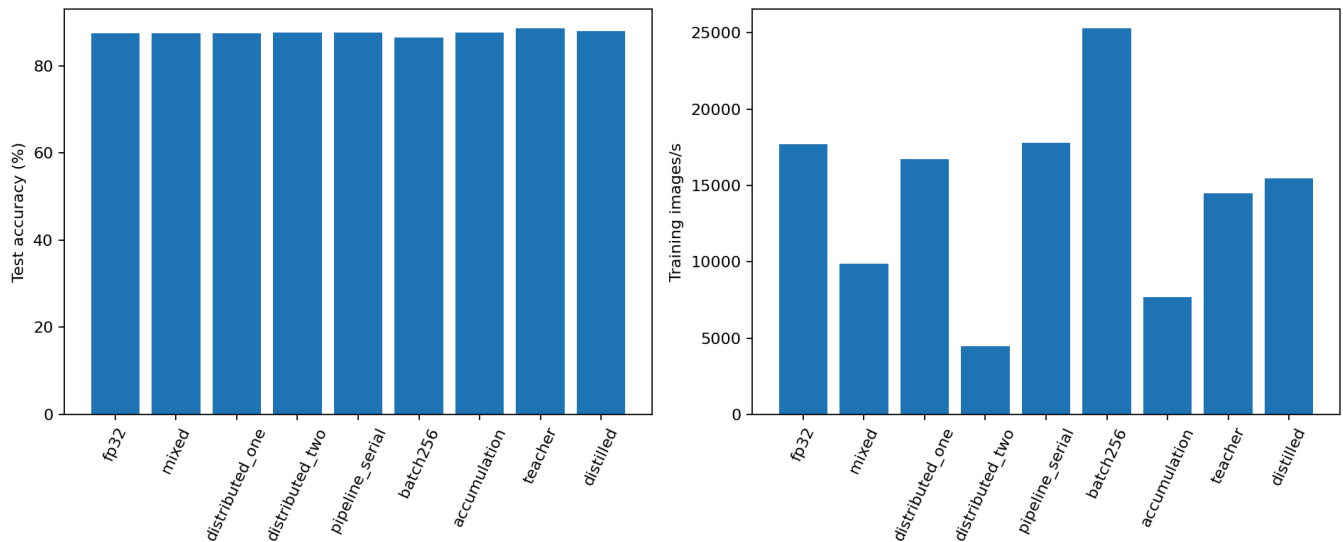


Figure 2. Comparison of cloud optimization experiments.

4. Edge Optimization Effectiveness

Pruning versus quantization. Pruning targets of 50% and 75% produced zero-value ratios of 49.61% and 74.42% across all stored weights, including BatchNorm state and biases. Their TFLite accuracies were 87.65% and 84.57%. Both dense TFLite files remained 1.238 MiB, with no consistent latency benefit. Higher sparsity improved compressed Keras storage, but did not automatically reduce dense deployment size.

In Part 3, dynamic range, INT8, and Float16 quantization produced files of 0.324, 0.328, and 0.625 MiB, with accuracies of 87.67%, 87.39%, and 87.72%. Quantization therefore provided clearer deployment storage savings than the tested pruning exports. Part 4 selected dynamic range quantization using validation accuracy, then evaluated the selected model on the test set.

Architecture optimization. Depthwise separable convolutions, reduced widths, ReLU6, and global max pooling reduced the fixed lightweight model to 15,157 parameters. Its Float32 TFLite accuracy was 70.54%. The eight-candidate NAS selected a model that reached only 67.96% after final training; this short search did not outperform the fixed architecture.

Part 4 combined the fixed lightweight model with INT8 quantization. Accuracy was 70.53%, and size fell from 67,880 to 36,248 bytes. This preserved the lightweight model's accuracy while reducing storage further. The measured Part 4 latency was 0.049 ms, versus 0.039 ms for its Part 3 Float32 version; these separate runs do not show an INT8 speed benefit.

TFLite conversion. Float32 conversion preserved baseline and pruned Keras accuracy within 0.03 percentage points. The exported models worked in the local interpreter. This confirms local conversion and inference, but deployment still requires target-device operator, memory, latency, and power checks.

5. Resource Constraints and Development Trade-offs

Cloud optimization emphasized throughput, with larger batches trading GPU memory for faster training. Edge optimization emphasized storage and accuracy, making quantization a useful balance. The microcontroller option required a much smaller architecture and accepted lower accuracy. A small model file alone does not guarantee that activation buffers fit in RAM or that power limits are met.

The complexity ratings reflect the assignment workflow. Cloud work was high because mixed precision, distributed synchronization, and distillation required different training paths. Edge work was medium because conversion reused trained models, although calibration and accuracy checks were needed. Microcontroller work was high because architecture changes, integer conversion, and device restrictions must be handled together. Reusing earlier models in Part 4 reduced repeated training and implementation work.

6. Deployment Recommendations

Scenario A: Real-time video processing. For the requested latency below 50 ms, high accuracy, and continuous operation, use the quantized edge model with cloud backup for difficult inputs. Prioritize accuracy preservation and target-device latency. The local 0.310 ms result covers only single-image model inference, so it does not prove that a complete video pipeline meets the requirement.

Scenario B: IoT sensor network. For power below 1 mW, months of operation, and periodic updates, use the lightweight INT8 model as a TinyML starting point with occasional cloud synchronization. Inference should be infrequent where the application allows. The Part 4 budget of 10 mW is higher than this scenario's limit, and consumption was not measured, so compliance cannot yet be claimed. Application-specific sensor data would also require retraining and evaluation.

Scenario C: Mobile application. For diverse devices and offline use, provide the quantized edge model locally and a smaller variant for more limited devices. Higher-capability devices or an available cloud connection can support more demanding processing. Prioritize an acceptable balance between accuracy, storage, and latency, and verify compatibility before app distribution.

7. Real-World Deployment and Future Evolution

To extend the pipeline for production, first measure complete inference memory, latency, and power on the intended hardware. Then test representative application inputs, supported operations, offline behavior, and model updates. A cascade could forward low-confidence predictions to a larger model, with thresholds selected on validation data.

Future hardware and runtime improvements could change these results. If low-precision operations become faster on the target hardware, mixed precision or INT8 may provide larger speed benefits. Better sparse execution could make pruning useful for latency as well as compression. The same benchmark process should be repeated when hardware or software changes, rather than assuming the current ranking will remain valid.

Results sources: the saved Part 1 baseline metrics, Part 2 cloud optimization report, Part 3 edge optimization report, and Part 4 multi-scale optimization report.